



nextwork.org

Build a RAG API with FastAPI



tahirgroot@gmail.com

<https://medium.com/@tahirbalarabe2/how-to-m...>



Introducing Today's Project!

In this project, I'm going to build a local RAG API using FastAPI, ChromaDB, and Ollama so I can ask personal questions and get accurate answers grounded in my own documents, not just the AI's training data. This will help me understand how retrieval-augmented generation works without relying on cloud costs or external APIs. I'm interested in this because I want to create AI assistants that answer from internal files like support tickets or personal notes, and I'm excited about extending it into a multi-user directory where anyone's profile can be queried.

Key tools and concepts

The key tools I used include FastAPI for building the API, ChromaDB as the vector database, Ollama for running local embedding and chat models like nomic-embed-text and qwen2.5:0.5b, plus Uvicorn as the server. Key concepts I learned include retrieval-augmented generation where I retrieve relevant context before generating answers, embeddings as numerical vectors for semantic search, chunking documents for better retrieval, metadata filtering for multi-user isolation, and building a complete RAG pipeline from document ingestion to query answering, all running locally with zero cloud costs.

Challenges and wins

This project took me approximately 30 minutes from start to finish, including the secret mission. The most challenging part was getting all the pieces to communicate correctly, especially making sure ChromaDB was using the right embedding function and that Ollama's server was running before pulling models. I also spent a few minutes debugging the metadata filter syntax for multi-user queries. Once the API was up and Swagger UI showed correct answers, everything clicked. The manual RAG demo in the beginning really helped me understand what the code needed to do before I even wrote it.



Performing RAG Manually

In this step, I'm going to run a manual RAG demo to see retrieval-augmented generation in action before coding. RAG stands for retrieval-augmented generation, a technique where I first retrieve relevant context from my documents, then generate an answer grounded in that context instead of relying solely on the AI's training data. After the demo, I will set up my Python project with a virtual environment, install all dependencies like FastAPI, ChromaDB, and Ollama, and pull the embedding model so my API can later answer personal questions accurately.



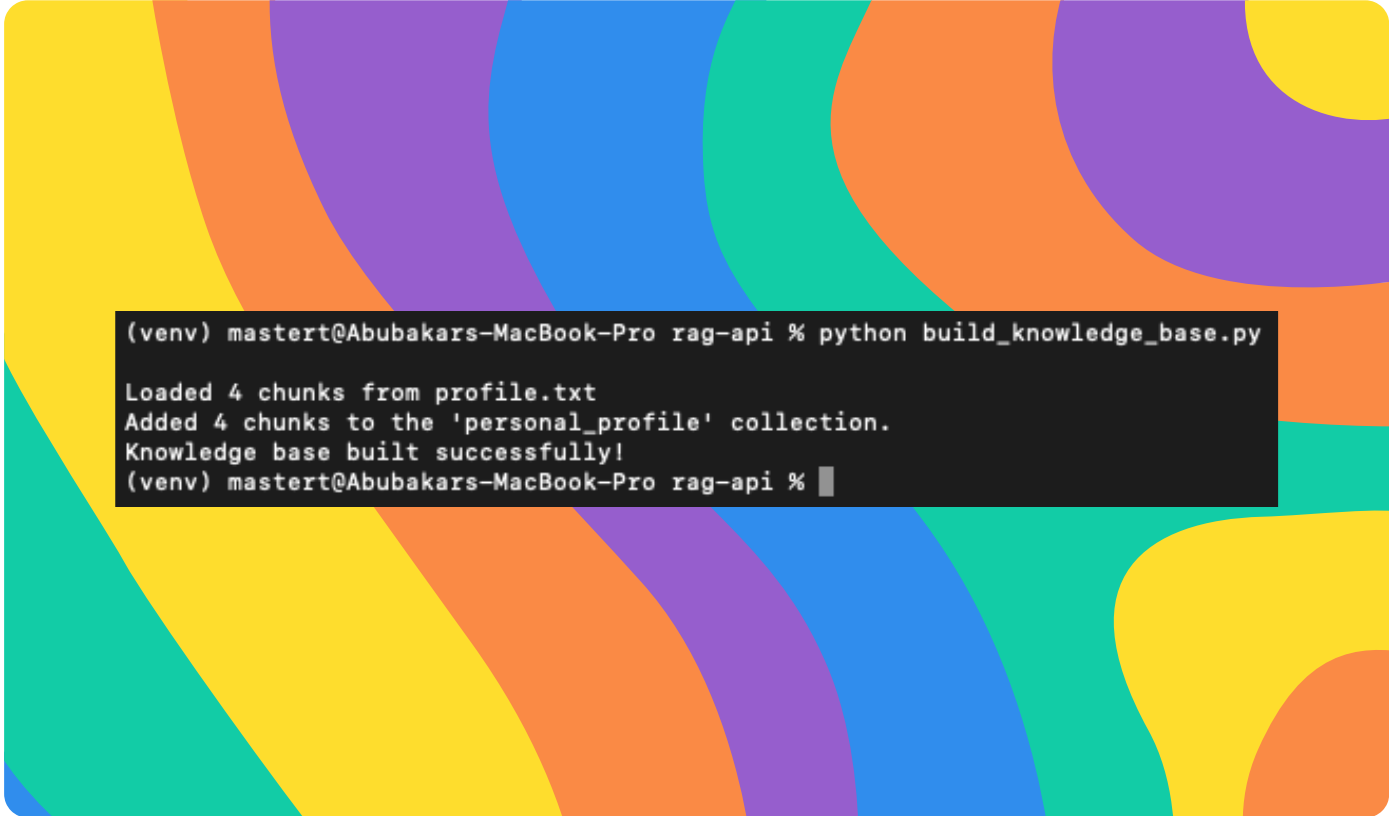
Understanding the three parts of RAG

I performed RAG manually by first retrieving relevant personal information about myself, then augmenting my prompt with that context, and finally having the AI generate a grounded answer. The three parts are retrieval, where I found the specific text about my career goals and interests; augmentation, where I added that text directly into my prompt before asking the question; and generation.



Building a Personal Knowledge Base

In this step, I'm going to build my knowledge base by creating a personal profile document, then writing a Python script that loads, chunks, and stores that profile as vector embeddings in ChromaDB. Embeddings are numerical representations of text that capture meaning, allowing the database to find relevant information through semantic search instead of keyword matching. This is the retrieval part of RAG. By converting my profile into embeddings, I give my AI a way to search for and retrieve personal context before generating answers. Once the script runs, I'll have a working local vector database ready for querying.

A terminal window screenshot is centered on a background of large, overlapping, colorful circles in shades of yellow, orange, purple, blue, and teal. The terminal text shows the execution of a Python script to build a knowledge base.

```
(venv) mastert@Abubakars-MacBook-Pro rag-api % python build_knowledge_base.py
Loaded 4 chunks from profile.txt
Added 4 chunks to the 'personal_profile' collection.
Knowledge base built successfully!
(venv) mastert@Abubakars-MacBook-Pro rag-api %
```

Creating the profile document

I included information about my name, current learning focus in cloud computing and AI, my specific career goal, areas of interest, existing experience, projects I'm building, and personal fun facts. This information will help the AI answer questions about me because without it, the model would have no knowledge of who I am, my background, or my aspirations. The profile acts as



Instead of guessing or hallucinating, the AI can pull directly from my profile to give accurate, personalized responses based on my actual life and goals.

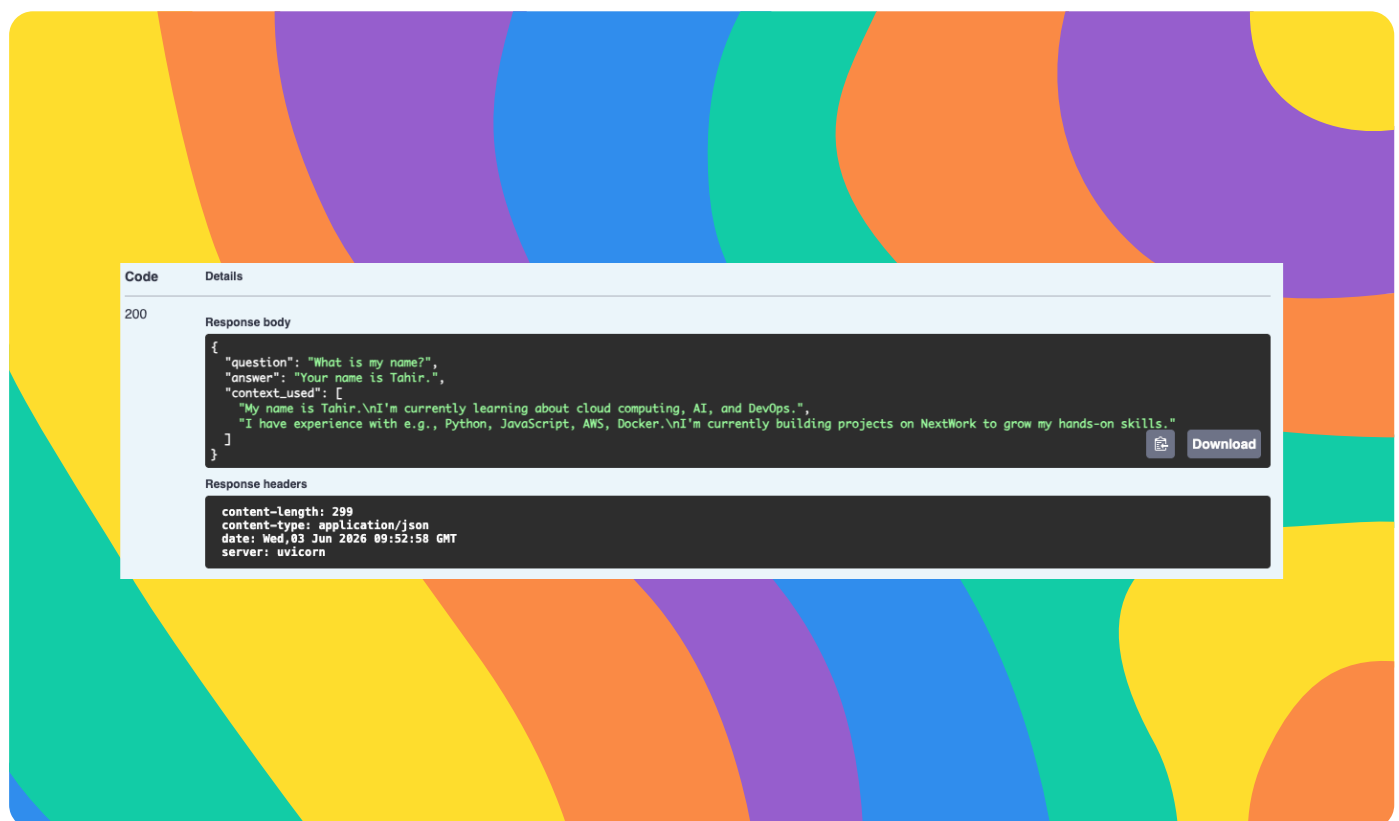
How semantic search finds relevant chunks

When I ask a question, ChromaDB first converts my question into a vector using the same nomic-embed-text model that was used to embed my profile chunks. Then it compares my question's vector to all stored chunk vectors by measuring distance in the high-dimensional space. Chunks whose vectors are closest to my question's vector are the most relevant. This is semantic search, not keyword matching. ChromaDB returns the closest chunks as context, which my API will pass to the chat model to generate a grounded answer based on my actual profile content.



Creating the RAG API with FastAPI

In this step, I'm going to build a FastAPI application with a /ask endpoint that implements the full RAG pipeline. When a question arrives, the API will automatically retrieve relevant chunks from my ChromaDB knowledge base, augment the prompt with that context, and call Ollama's qwen2.5:0.5b model to generate a grounded answer. I'll test the API using FastAPI's built-in Swagger UI documentation page at /docs, which lets me send questions and see responses directly from my browser without writing any frontend code. This gives me a working local RAG API.



How the /ask endpoint works

When a question comes in, my endpoint first retrieves the two most relevant chunks from ChromaDB by converting the question into a vector and finding semantically similar profile chunks. Second, it augments the prompt by combining those retrieved chunks with the original question into a structured prompt that includes context. Third, it generates an answer by sending that



Finally, the endpoint returns the question, the answer, and the context used for verification.

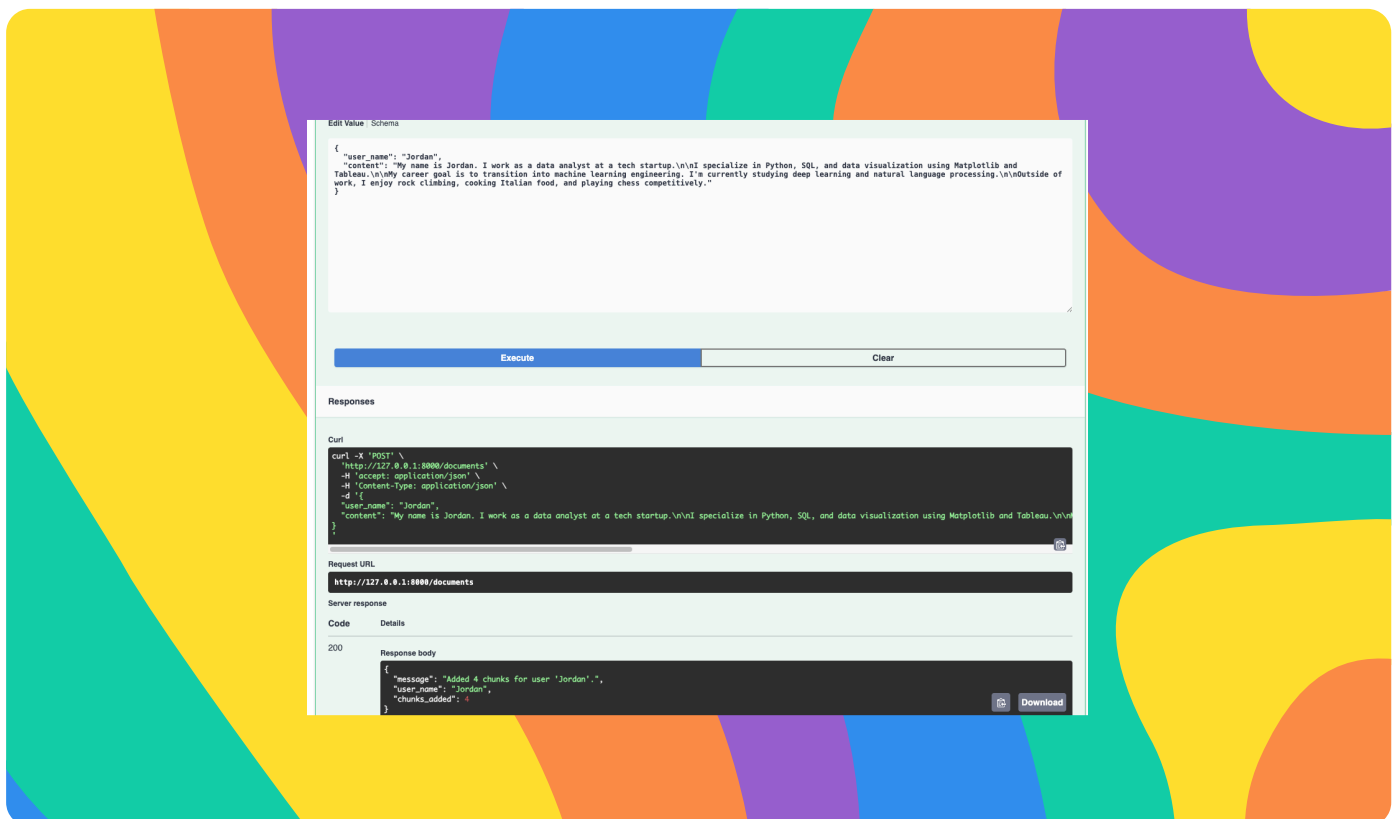
Testing with Swagger UI

I tested my API by asking “What is my name?” The AI answered with my actual name from my profile. The context used was the exact chunk from profile.txt that contained my name, because ChromaDB retrieved that semantically relevant chunk. Without that retrieved context, the AI would have guessed or said it didn’t know me. Instead, the RAG pipeline automatically found the right paragraph, augmented the prompt with it, and generated a grounded, accurate answer based solely on my personal document. This confirmed that my local RAG API works end to end.



Extending to a Multi-User AI Directory

In this project extension, I'm adding multi-user support because real-world RAG systems almost always serve many users or data sources simultaneously, like enterprise knowledge bases or customer support bots. Multi-tenancy means isolating data so each user only queries their own documents, even though they share the same vector database. This teaches me how to handle metadata filtering, separate storage per user, and build APIs that support multiple tenants. Without multi-user support, my API would mix everyone's profiles together, causing incorrect answers. Adding it makes my RAG API scalable and production ready for real applications.

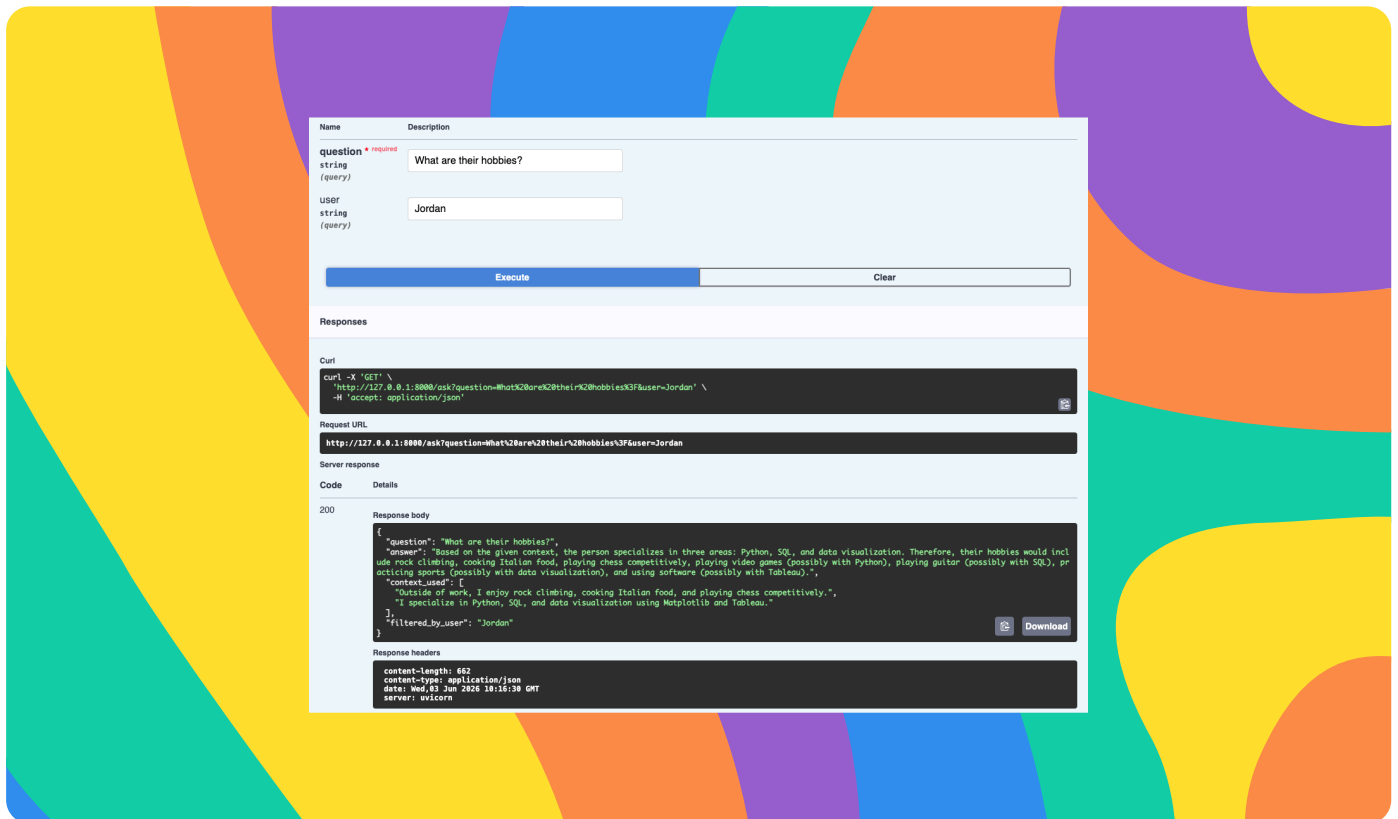


Adding the POST /documents endpoint

In this project extension, I added a POST endpoint that accepts a `user_name` and profile content, splits the content into chunks, and stores each chunk in ChromaDB with `user_name` attached as metadata. Metadata filtering allows the GET /ask endpoint to search only chunks belonging to a specific user by adding a



Without metadata filtering, queries would mix profiles together and produce wrong answers.



Verifying multi-user filtering

In this project extension, I tested multi-user queries by first asking “What are their hobbies?” with no user filter, which returned mixed chunks from my profile and Jordan’s profile. Then I asked the same question with user=Jordan, and the context_used contained only Jordan’s hobbies like rock climbing and chess, while the answer was accurate and not mixed with my data.



tahirgroot@gmail.com

NextWork Student

nextwork.org

The filter works because ChromaDB's "where" parameter limits the semantic search to only chunks whose user_name metadata matches the provided value. This prevents cross-user contamination and ensures each user only queries their own profile.



Wrapping Up

I did this project today to learn how to build a local RAG API from scratch using FastAPI, ChromaDB, and Ollama so I could answer personal questions with grounded AI responses without sending my data to the cloud. Another skill I want to learn is implementing authentication and authorization, like API keys or JWT tokens, to turn this multi-user directory into a secure production system where users can only query their own private profiles. I also want to explore different chunking strategies and larger embedding models to improve retrieval accuracy.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

